

마이크로프로세서개론 프로젝트 보고서

서강대학교 전자공학과

20181501 김준원

20181503 김찬구

1. 목적

: 마이크로프로세서응용실험(EEE4183)에서 배운 내용(Interrupt, timer, USART 등)을 바탕으로 주어진 문제를 제공하는 Nucleo 보드와 소자들을 이용하여 구현한다.

2. 단계별 구현 방법

본 과제의 단계 3을 구현하였으며 기능별 구현 방법은 아래와 같다.

i) Dot matrix scanning & scrolling & 밝기 제어

디스플레이 제어를 위한 각 board별 역할 및 동작은 다음과 같다.

Board1: 동작 Mode 제어 및 Display 출력내용 준비

FLASH에 저장된 이전 mode 값 및 설정을 불러온다.

USART를 통해 전송할 출력 내용을 준비한다.

➤ 출력 내용 (8byte): brightness(1 byte), digits(5 byte), n_shift(1byte), state(1byte)

Brightness: ADC를 통해 측정한 조도 값을 토대로 한 10~255 사이의 화면 밝기 값

Digits: '0'~'9', 'l', 'x', 'F', 'C', '°'(degree), '.'(dot), ':'(colon), ' '(space)

State: Bit[2] – 오전 / 오후, Bit[1] – 12시간제 / 24시간제, Bit[0] – 초기시간 설정여부

N_shift: 0~3의 값을 가지며 화면에서 digits가 왼쪽으로 n칸 shift 했음을 나타냄

40Hz Timer interrupt마다 ADC 값(온도/조도)을 저장하고 USART를 통해 출력내용을 전송

8Hz Timer interrupt마다 display로 출력할 window를 shift 시킨 후 brightness를 update함

1Hz Timer interrupt마다 Base_time을 1초 증가시킨 후 출력할 온도/조도 값을 update함

Board2: USART 통신을 통해 전달받은 Display 출력내용을 토대로 Dot matrix 제어

8 byte의 출력내용을 USART로 수신 후 Dot matrix 출력 패턴 형성

- *Digits*에 해당하는 5x3 dot matrix pattern 정보를 font5x4 변수에 저장

- *Digits*[0]~*digit*[4]의 내용을 32 bit 변수에 저장 후 이를 왼쪽으로 *n_shift*만큼 shift함

8Hz의 Timer interrupt마다 TIM->CCR2 값 update 통해 화면 밝기 조절

960Hz의 Timer의 CC1에서(CNT == 0) Dot matrix의 다음 Row값을 출력

960Hz의 Timer의 CC2에서(CNT == ARR*brightness/255) Dot matrix의 Row를 소등

따라서, dot matrix는 120Hz의 주사율을 가지며 동작을 하고, 960Hz timer의 CC1과 CC2의 발생 간격 차이를 통해 화면의 밝기를 조절한다.

또한, Board1에서 8Hz 주기로 *n_shift* 및 *disp_idx*를 update하여 화면 scrolling 기능을 수행한다.

ii) Key matrix scanning & debouncing 제어

Board1에서 다음과 같은 방식을 통해 Key matrix의 scanning과 debouncing, 그리고 Key 입력에 따른 동작 *mode* 변경을 수행한다.

960Hz의 Timer interrupt마다 Key matrix의 한 row에 대해 scanning을 수행함.

→ Key matrix의 col값에 따라 입력된 key의 개수 n_key 를 증가시킴

4개 row에 대한 scan이 완료된 경우 입력된 key 개수 아래의 동작을 수행함.

$n_key == 0$: *Key_state* 변수에 0xFF를 저장하여 아무런 Key 입력이 없음을 나타냄

$n_key == 1$: *Key_state* 변수에 유일하게 눌린 key의 index(0~15)를 저장

$n_key >= 2$: *Key_state* 변수값을 update 하지 않음 (Key 중복입력의 경우 입력 무시)

→ 입력이 감지되면($n_key >= 1$) TIM2→CCR1값을 update하여 key 입력 대기시간 초기화

→ $n_key == 1$ 의 유효한 입력이 인가된 경우 EXTI0 인터럽트를 S/W적으로 발생

EXTI0 인터럽트가 발생 시(유효한 입력 인가) *mode* 및 각종 변수들을 변경

mode != Time_Displ인 경우 다음에 입력에 대한 동작을 수행함.

Key_state == 0: *mode* 값을 Temp_disp로 변경

Key_state == 1: *mode* 값을 Temp_disp로 변경

Key_state == 2: *mode* 값을 Illu_disp로 변경

Key_state == 3:

Time_disp → Time_Set 모드로 전환,

Temp_disp 모드 → 섭씨/화씨 전환

mode == Time_Set : 시간 설정 모드에서 다음의 입력이 인가되면
우측 그림과 같이 동작함.

- 초 설정에서 확인을 눌렀을 시 전체 시간 변경됨
- UP/DOWN 키 및 키패드를 통한 시 분 초 설정 가능



iii) 시계 및 시간 표시 기능

Board1에서 시계 기능을 수행하며 *mode*에 따라 시간 표시형식에 맞는 출력내용을 설정함.

1Hz의 Timer interrupt에 맞추어 *Base_Time* 변수의 값을 1 증가시킴

States 변수의 설정값에 따라 시간 출력 형식을 조절 (12시제 - 0~11시, 24시제 - 0~23시)

iv) 온도 및 조도 측정 및 표시 기능

Board1에서 ADC를 통해 온도 및 조도센서의 값을 측정 후 형식에 맞게 출력내용을 설정함

40Hz Timer 신호에 맞춰 ADC를 trigger 후 변환이 완료되면 이를 Callibration으로 얻은 수식을 통해 섭씨/화씨, lux 값으로 변환하여 저장함

➢ TMP36: 온도에 따른 전압특성이 선형성을 보임 → 기준치와의 편차를 통해 온도값 계산

➢ GL5536: 조도에 따른 특성이 비선형적 특성을 보임 → 기준치를 통해 LUT을 형성 후
기준치 사이의 값은 linear interpolation을 통해 조도값을 추정함

v) USART 통신 기능

Tx-Rx, CTS-RTS, GND를 연결하여 Hardware flow control이 가능한 Simplex mode로 동작함.

- Board1(B1): 40Hz마다 첫 데이터 전송 및 CTS 인터럽트가 발생 시 8회까지 데이터 추가 전송
- Board2(B2): Tx-Rx라인이 안정되었다고 판단 시 RTS 신호를 발생하여 데이터 수신

i) Tx-Rx 라인 연결 해제 시

B2: frame error → B2: Disconnect mode (RTS disable) → B1: CTS 전달 안 됨

→ B1: 데이터 전송 횟수 8회 미만 → B1: Break Character 전송 (disconnect, *IsConnect* = 0)

→ B1: 40Hz 주기로 Test 신호 전송하며 선로 복구 여부 확인

ii) Tx-Rx 라인 연결 복구 시

B2: 40Hz Test 신호 수신 → B2: Test 신호가 안정적으로 6회 수신

→ B2: Handshake mode (RTS enable) → B1: CTS 신호 수신에 따른 Test 신호 8회 전송

→ B1: 선로에 이상 없다고 판단 후 출력 내용 전송 (connected, *IsConnect* = 1)

→ B2: Test 신호가 아닌 내용이 전송됨에 따라 순차적 데이터 저장 시작(Disp_On mode)

iii) CTS-RTS 라인 연결 해제 시

B1: CTS 전달 안됨 → B1: 데이터 전송 횟수 8회 미만 → B1: Break character 전송(disconnect)

→ B2: break character 수신에 따른 Frame error → B2: RTS disable (Disconnect mode)

→ B1: 40Hz 주기로 Test 신호 전송하며 선로 복구 여부 확인

→ B2: Test 신호 6회 이상 수신에 따른 Handshake mode (RTS enable)

iv) CTS-RTS 라인 연결 복구 시

→ B1: CTS 신호 수신 → B1: Test 신호 8회 전송 → B1: 출력 내용 전송 (connected)

→ B2: Test 신호가 아닌 내용이 전송됨에 따라 순차적 데이터 저장 시작 (Disp_On mode)

vi) FLASH 기능

Key matrix에서 *mode*, 섭씨/화씨 설정이 변경된 경우 Flash에 *states* 변수를 저장함.

이후 reset 시 main function에서 state 변수값을 통해 이전 mode 및 설정을 불러옴

3. 코드 설명

i) Board1

Main & Tx_Set function		
<pre> 140 /*-----Initial State setting-----*/ 141 states = (uint8_t)FLASH_ADDR; // Read previous states 142 states = 0xFF00; 143 states = 0x0004; // Clock undefined & P.M. 144 states = 0x0001; 145 states = (states & 0x0007) & 0x01; // Restore 12/24 setting 146 147 #switch(states >> 12){ 148 case 2: 149 mode = Temp_Disp; 150 break; 151 case 3: 152 mode = Illu_Disp; 153 break; 154 default: 155 mode = Time_Set; 156 break; 157 } 158 159 //-----Main function-----*/ 160 while(1){ 161 // Check UART Communication 162 while (IsConnect == 0){ 163 TIM3->CCR1 = 0; // Key matrix off 164 TIM3->CCR1 = 0; // Key matrix on 165 _WFI(); 166 if (IsConnect){ 167 TIM3->CCR1 = 0x01; // Key matrix on 168 if (mode == Time_Set){ 169 TIM2->DIER = 0x02; 170 } 171 } 172 } 173 174 if (mode == Time_Set){ 175 B[0] = (states & 0x0007) & Base_Time_HH & 12; 176 B[1] = B[0] & 10; 177 B[2] = B[0] / 10; 178 B[3] = Base_Time_MM & 10; 179 B[4] = Base_Time_SS & 10; 180 B[5] = Base_Time_SS / 10; 181 S[0] = Base_Time_SS & 10; 182 S[1] = Base_Time_SS / 10; 183 184 digit[0] = SP; 185 digit[1] = SP; 186 digit[2] = H[1]; 187 digit[3] = H[0]; 188 digit[4] = column; 189 digit[5] = H[2]; 190 digit[6] = H[3]; 191 digit[7] = column; 192 digit[8] = S[1]; 193 digit[9] = S[0]; 194 n_digit = 10; 195 196 Tx_Set(); 197 _WFI(); 198 } </pre>	<pre> 219 else if (mode == Temp_Disp){ 220 B[0] = Disp_Temp / 1000; 221 B[1] = Disp_Temp / 100 & 9; 222 B[2] = Disp_Temp / 10 & 9; 223 B[3] = Disp_Temp - B[3]*1000 - B[2]*100 & 9; 224 225 digit[0] = SP; 226 digit[1] = SP; 227 digit[2] = B[3]; 228 digit[3] = B[2]; 229 digit[4] = B[1]; 230 digit[5] = B[0]; 231 digit[6] = SP; 232 digit[7] = SP; 233 digit[8] = (states & 0x0007) & F & 1; 234 n_digit = 9; 235 236 Tx_Set(); 237 _WFI(); 238 } 239 240 else if (mode == Illu_Disp){ 241 B[0] = Disp_Ill / 1000; 242 B[1] = Disp_Ill / 100 & 9; 243 B[2] = Disp_Ill / 10 & 9; 244 B[3] = Disp_Ill - B[3]*1000 - B[2]*100 & 9; 245 246 digit[0] = SP; 247 digit[1] = SP; 248 digit[2] = B[3]; 249 digit[3] = B[2]; 250 digit[4] = B[1]; 251 digit[5] = B[0]; 252 digit[6] = SP; 253 digit[7] = SP; 254 n_digit = 9; 255 256 Tx_Set(); 257 _WFI(); 258 } </pre>	<pre> 284 case 2: // Second setting 285 digit[0] = column; 286 digit[1] = (states & 0x01) & S[1]; 287 digit[2] = (states & 0x01) & S[0]; 288 digit[3] = SP; 289 break; 290 291 case 3: 292 n_digit = 4; 293 Tx_Set(); 294 _WFI(); 295 } 296 297 //-----User Defined Function-----*/ 298 void Tx_Set(void){ 299 tx_data[0] = brightness; 300 tx_data[1] = digit[(disp_idx & 1) & n_digit]; 301 tx_data[2] = digit[(disp_idx & 1) & n_digit]; 302 tx_data[3] = digit[(disp_idx & 2) & n_digit]; 303 tx_data[4] = digit[(disp_idx & 3) & n_digit]; 304 tx_data[5] = digit[(disp_idx & 4) & n_digit]; 305 tx_data[6] = n_shift; 306 tx_data[7] = (uint8_t)(states & 0xFF); 307 } </pre>

	<pre> 259 else if (mode == Time_Set) { 260 M[1] = (states & 0x03) ? Mod_Time_HH : Mod_Time_HH % 12; 261 M[0] = M[1] % 10; 262 M[3] = M[1] / 10; 263 M[2] = Mod_Time_MM / 10; 264 M[0] = Mod_Time_MM % 10; 265 M[1] = Mod_Time_SS / 10; 266 M[3] = Mod_Time_SS % 10; 267 268 if (Mod_Time_HH / 12) 269 states = 0x01; 270 else 271 states &= ~(0x01); 272 273 switch (Mod_Time) { 274 case 0: // Hour setting 275 digits[0] = HF; 276 digits[1] = (states & 0x01)? M[1]: SF; 277 digits[2] = (states & 0x01)? M[0]: SF; 278 digits[3] = colon; 279 break; 280 case 1: // Minute setting 281 digits[0] = colon; 282 digits[1] = (states & 0x01)? M[1]: SF; 283 digits[2] = (states & 0x01)? M[0]: SF; 284 digits[3] = colon; 285 break; </pre>	
--	---	--

Main function에서는 Flash에 저장된 설정값을 저장 후 각 mode별 display할 내용 및 화면에 출력할 window (disp_idx, n_shift)에 맞추어 tx_data를 설정하는 역할을 수행한다.

Line 161~171은 Flash 데이터를 읽고 이를 State에 반영하는 내용으로 Reset 후 시간은 12:00:00가 되며 시간 미설정 상태로 *states*변수를 설정한다. Line 185~194의 경우 *IsConnect* 변수가 0으로 board2와의 연결이 불량할 시 key matrix를 제어하는 TIM2를 disable 하여 key matrix의 입력을 받지 않도록 하였다. Line 195~298은 각 모드별 출력화면을 구성하는 동작을 수행하며, 전체 출력 내용을 digits에 저장 후 Tx_set함수를 호출하여 disp_idx 및 n_shift를 토대로 digits의 일부를 tx_data[1]~tx_data[5]에 저장하게 된다. 이후 _WFI()함수를 통해 CPU가 동작하지 않을 경우 전력소모를 최소화하였다.

ADC1_2 ISR

```

313 void ADC1_2_IRQHandler(void) { // TIM3, GL5537 Calibration & brightness update
314     if (TIM3->SR & 0x001) { // EOC, channel_1(TMP36)
315         ADC_TMP = ADC1->SR;
316         ADC1->SR &= ~(0x001);
317         ADC_TMP = (states & 0x001)? (ADC_TMP*10/50 + 572) : (ADC_TMP/5 + 140); // Fahrenheit to Celsius
318     }
319     if (ADC1->SR & 0x004) { // JEOC, channel_0(GL5537)
320         ADC_IL1 = ADC1->NDRL1;
321         ADC1->SR &= ~(0x004);
322         if (ADC_IL1 < 0x40)
323             ADC_IL1 = 100;
324         else if (ADC_IL1 < 0x800)
325             ADC_IL1 = 100 + (100-10) * (ADC_IL1*10 - 0x40*10) / (0x800-0x40);
326         else if (ADC_IL1 < 0x8000)
327             ADC_IL1 = 1000 + (1000-100) * (ADC_IL1*10 - 0x800*10) / (0x8000-0x800);
328         else if (ADC_IL1 < 0x80000)
329             ADC_IL1 = 2000 + (2000-1000) * (ADC_IL1*10 - 0x800*10) / (0x80000-0x800);
330         else if (ADC_IL1 < 0x800000)
331             ADC_IL1 = 4000 + (4000-1000) * (ADC_IL1*10 - 0x800*10) / (0x800000-0x800);
332         else
333             ADC_IL1 = 8000;
334     }
335 }

```

해당 인터럽트는 ADC conversion이 완료됐을 때 수행되며, ADC를 통해 변환된 값을 섭씨/화씨, 그리고 lux값으로 변경하여 저장하는 역할을 수행한다(24.5°C의 경우 245를 저장한다). 코드에서 사용된 수식은 실험적으로 설정한 기준값에 의한 calibration 코드이다.

TIM1 UPDATE ISR

```

347 void TIM1_UP_IRQHandler(void) { // 8Hz - Scrolling & Blinking & Brightness
348     if (TIM1->SR & 0x001) {
349         if (IsConnect == 0) // if Display disconnected
350             GPIOA->ODR ^= (1u << 5); // LED blink
351     }
352     else {
353         brightness /= 2;
354         brightness += ADC_IL1 * 120 / 8000 + 5;
355
356         if (mode != Time_Set) { // Scrolling
357             n_shift++;
358             if (n_shift >= 4) {
359                 disp_idx = 0;
360                 disp_idx++;
361             }
362             if (disp_idx >= n_digit)
363                 disp_idx = 0;
364             if ((states & 0x0000)) // If Clock unset
365                 states ^= 0x01; // Dot blink
366         }
367         else // If mode == Time_set
368             states ^= 0x01; // Dot blink
369     }
370     TIM1->SR &= ~(0x001);
371 }

```

8Hz의 주기를 갖는 인터럽트로 Board2가 연결되지 않을 시 PA5에 연결된 LD2(Green LED)가 깜빡이도록 한다. Board2가 연결되었을 경우, Dot matrix scrolling에 필요한 n_shift 및 n_digit을 update하고, 화면 출력 밝기인 brightness를 update해주었다. 이때, ADC값과 기존 brightness의 평균값을 저장함으로써 10~255사이에서 brightness가 부드럽게 변할 수 있도록 하였다. 또한, 초기시간이 설정되지 않았거나 Time_Set 모드로 동작하는 경우 Dot matrix의 우측하단에 해당하는 *states*변수의 bit[0]를 toggle시킨다.

TIM1 CAPTURE/COMPARE ISR

```

373 void TIM1_CC_IRQHandler(void) { // 40Hz - ADC & USART trigger
374     if (TIM1->SR & 0x002) {
375         ADC1->CR2 |= (0x1 << 22); // ADC1 - SWSTART, JWSWSTART = '1' : conversion start
376         if (n_transmit >= 8) {
377             IsConnect = 1;
378             GPIOA->ODR ^= (1u << 5);
379             USART1->DR = tx_data[0];
380             n_transmit = 0;
381         }
382         else { // Previous tx_data sending : Error
383             if (IsConnect) {
384                 USART1->CR1 |= 0x01; // SBK set : Send break character
385             }
386             else if (USART1->SR & 0x0000) { // TXE = '1'
387                 USART1->DR = TEST_DATA;
388                 USART1->CR3 |= 0x0020; // CTSE set;
389             }
390             else {
391                 USART1->DR = TEST_DATA;
392                 USART1->CR3 |= 0x0200; //CTSE reset;
393             }
394             n_transmit = 0;
395             IsConnect = 0;
396         }
397     }
398     TIM1->SR &= ~(0x002);
399 }
400 }

```

40Hz 주기를 갖는 인터럽트로 ADC의 SWSTART(regular channel for TMP36)과 JWSTART(injected channel for GL5537)를 set하여 ADC변환을 시작한다.

또한, 이전 전송횟수를 통해 이전 전송에서의 문제여부를 판단한다. 만일 처음으로 문제가 감지되었다면 break character를 전송하게 되며, 그렇지 않을 경우 테스트 데이터(0)를 전송한다. 만일 CTS로 인해 TDR 레지스터가 차 있는 경우 강제적으로 CTS 기능을 잠시 Disable 해주어 지속적으로 테스트 데이터를 전송할 수 있도록 한다.

TIM2 ISR

<pre> 402 void TIM2_IRQHandler(void) { // 1Hz - 1Hz : Clock & ADC Display update, CH1 : 3 seconds violation 403 if (TIM2->SR & 0x001) { // TIM2_UP 404 Base_Time.SS++; 405 if (Base_Time.SS >= 60){ 406 Base_Time.SS -= 60; 407 Base_Time.MM++; 408 } 409 if (Base_Time.MM >= 60){ 410 Base_Time.MM -= 60; 411 Base_Time.HH++; 412 } 413 if (Base_Time.HH >= 12){ 414 states = 1 << 2; 415 if (Base_Time.HH >= 24){ 416 Base_Time.HH -= 24; 417 states &= ~(1 << 2); 418 } 419 // ADC display update 420 Disp_THR = ADC_THR; 421 Disp_TLL = ADC_TLL; 422 TIM2->SR &= ~(0x001); 423 } 424 } 425 else if ((TIM2->SR & 0x002) != 0){ // TIM2_CH1 426 Key_cnt++; 427 if (Key_cnt >= 3) { 428 mode = Time_Disap; 429 Mod_cur = 0; 430 Key_cnt = 0; 431 states &= 0x00FF; 432 states = (1 << 12); 433 } 434 TIM2->SR &= ~(0x002); 435 } 436 } </pre>	1Hz로 발생하는 인터럽트로 TIM2 update에 의해 인터럽트가 발생했을 시 <i>Base_Time</i>을 1초 더하게 되며, 이 과정에서 초, 분, 시간이 각각 60초, 60초, 24시간이 되었을 경우 올림을 수행하며, Display에 출력할 ADC값을 update해주어 1Hz단위로 화면 출력이 변경되도록 하였다. Output Compare 모드로 동작하는 TIM2_CH1에 의해 인터럽트가 발생했을 시 이는 Key 입력이 없는 채로 1초가 경과하였음을 의미하는 것이므로 3번의 인터럽트가 발생 시 Time_disp mode로 돌아가도록 하였다.
TIM3 ISR	
<pre> 438 void TIM3_IRQHandler(void) { // 960Hz - Key matrix scanning 439 static u16 key_row = 0, key_col = 0, n_key = 0, key_idx = 0xFF; 440 static u16 CNT = 0; 441 if (TIM3->SR & 0x001) { // TIM3_UP 442 key_col = GPIOB->IDR; // Get Key matrix column inputs 443 key_col = (key_col >> 8) & 0x0F; // Masking Key column inputs 444 switch(key_col) { 445 // Identifying inputs 446 // No inputs 447 case 0: 448 break; 449 case 1: // Col[0] 450 CNT = TIM2->CNT; 451 key_idx = 4*key_row; 452 n_key++; 453 break; 454 case 2: // Col[1] 455 CNT = TIM2->CNT; 456 key_idx = 4*key_row + 1; 457 n_key++; 458 break; 459 case 3: // Col[2] 460 CNT = TIM2->CNT; 461 key_idx = 4*key_row + 2; 462 n_key++; 463 break; 464 case 4: // Col[3] 465 CNT = TIM2->CNT; 466 key_idx = 4*key_row + 3; 467 n_key++; 468 break; 469 default: // Multiple inputs 470 CNT = TIM2->CNT; 471 n_key += 2; 472 } 473 Key_row++; 474 if (key_row == 4) { // Scanning over 16 Keys: completed 475 if (n_key == 0) 476 Key_state = Key_IDR; // Key_state clear 477 else if (n_key == 1) 478 Key_state = Key_IDR; 479 else if (n_key > 1) 480 EXTIO->SR = 0x001; 481 TIM2->CCR1 = (CNT) ? CNT - 1 : TIM2->ARR; 482 Key_cnt = 0; 483 } 484 else { 485 TIM2->CCR1 = (CNT) ? CNT - 1 : TIM2->ARR; 486 Key_cnt = 0; 487 } 488 Key_idx = 0xFF; 489 Key_row = 0; 490 n_key = 0; 491 CNT = 0; 492 } 493 GPIOC->ARR = (1 << (key_row * 5)) (1 << (key_row * 24)) & 0x0F000000; // Turn on next key row 494 TIM3->SR &= ~(0x001); 495 } </pre>	960Hz로 발생하는 인터럽트로 Key matrix의 scan을 담당한다. 한 번의 인터럽트 당 key matrix 한 row에 대해 scan을 진행하며, 각 row별 입력된 key 개수를 <i>n_key</i> 변수에 추가하고 하나의 입력이 있을 경우에만 <i>key_idx</i>를 update한다. 또한, 각 key가 눌렸을 경우, 그 때의 TIM2의 CNT 값을 저장하고, 다음 row를 활성화한다. 다음 row를 활성화 한 후 다음 cycle에서 해당 row의 입력을 확인함으로써 software적인 debouncing을 구현하였다. 그렇게 4개의 row에 대해 scan이 끝이 난 경우, 4개의 row를 scan하며 인가된 key 개수가 1개이하일 때, <i>Key_state</i>값을 update하고, 1개 이상일 때, TIM2->CCR2 레지스터에 CNT값을 저장하여 각 key가 입력되고 1초가 경과되면 TIM2에서 Compare interrupt가 발생되도록 한다. 또한 1개의 key만 입력되었을 때 그 key가 새로이 인가된 key라면(처음 key가 눌렸을 때), EXTIO interrupt를 S/W적으로 발생시켜 해당 key에 따른 동작을 수행하도록 하였다. 이후 <i>key_idx</i>, <i>key_row</i>, <i>n_key</i>, CNT 등의 변수를 초기화 해준다.
EXTIO ISR	
<pre> 498 void EXTIO_IRQHandler(void) { // Valid Key input 499 static u8 num; 500 if (EXTIO->FR & 0x001) { 501 if (mode == Time_Disap) { 502 TIM2->OIER &= ~(0x002); 503 switch (Key_state) { 504 case 0: 505 EXTIO->FR = 0x001; 506 return; 507 case 1: 508 mode = Temp_Disap; 509 states &= 0x00FF; 510 states = (1 << 12); 511 break; 512 case 2: 513 mode = Tllu_Disap; 514 states &= 0x00FF; 515 states = (1 << 12); 516 break; 517 case 3: 518 mode = Time_Set; 519 TIM2->OIER = 0x002; 520 Mod_Time.HH = Base_Time.HH; 521 Mod_Time.MM = Base_Time.MM; 522 Mod_Time.SS = Base_Time.SS; 523 EXTIO->FR = 0x001; 524 disp_idx = 0; 525 n_shlft = 0; 526 return; 527 } 528 } 529 else if (mode == Temp_Disap) { 530 TIM2->OIER &= ~(0x002); 531 switch (Key_state) { 532 case 0: 533 mode = Time_Disap; 534 states &= 0x00FF; 535 states = (1 << 12); 536 break; 537 case 1: 538 EXTIO->FR = 0x001; 539 return; 540 case 2: 541 mode = Tllu_Disap; 542 states &= 0x00FF; 543 states = (1 << 12); 544 break; 545 case 3: 546 states = (1 << 10); 547 Disp_THR = ADC_THR; 548 break; 549 } 550 } 551 } </pre>	<pre> 552 case 2: // Down 553 switch (Mod_cur) { 554 case 0: 555 Mod_Time.HH = (Mod_Time.HH ? Mod_Time.HH - 1 : 23); 556 break; 557 case 1: 558 Mod_Time.MM = (Mod_Time.MM ? Mod_Time.MM - 1 : 59); 559 break; 560 case 2: 561 Mod_Time.SS = (Mod_Time.SS ? Mod_Time.SS - 1 : 59); 562 break; 563 } 564 EXTIO->FR = 0x001; 565 return; 566 case 3: // Confirm 567 Mod_cur++; 568 if (Mod_cur < 3) { 569 EXTIO->FR = 0x001; 570 return; 571 } 572 Mod_cur = 0; 573 Key_cnt = 0; 574 Base_Time = Mod_Time; 575 mode = Time_Disap; 576 states &= 0x00FF; 577 states = 0x001; 578 states &= ~(0x001 << 9); 579 break; 580 case 7: // Right 581 Mod_cur = (Mod_cur + 1) % 3; 582 break; 583 case 11: // 0 584 switch (Mod_cur) { 585 case 0: 586 Mod_Time.HH = (HH[0] * 10) % 24; 587 break; 588 case 1: 589 Mod_Time.MM = (MM[0] * 10) % 60; 590 break; 591 case 2: 592 Mod_Time.SS = (SS[0] * 10) % 60; 593 break; 594 } 595 break; 596 case 15: // Left 597 if (Mod_cur == 0) 598 states = 0x0002; 599 else 600 Mod_cur--; 601 break; </pre>

<pre> 552 else if (mode == Illu_Displ) 553 TIM2->DIER &= ~(0x01); 554 switch (Key_state){ 555 case 0: 556 mode = Time_Displ; 557 states &= 0x0FFF; 558 states = (1 << 12); 559 break; 560 case 1: 561 mode = Temp_Displ; 562 states &= 0x0FFF; 563 states = (2 << 12); 564 break; 565 case 2: 566 EXTI->PR = 0x01; 567 return; 568 case 3: 569 EXTI->PR = 0x01; 570 return; 571 } 572 else if (mode == Time_Set) { 573 switch (Key_state){ 574 case 0: // Up 575 switch (Mod_cur) { 576 case 0: 577 Mod_Time.HH = (Mod_Time.HH + 1) % 24; 578 break; 579 case 1: 580 Mod_Time.MM = (Mod_Time.MM + 1) % 60; 581 break; 582 case 2: 583 Mod_Time.SS = (Mod_Time.SS + 1) % 60; 584 break; 585 } 586 EXTI->PR = 0x01; 587 return; 588 case 1: // Ignore 589 EXTI->PR = 0x01; 590 return; 591 default: 592 // 1 - 9 593 num = (Key_state % 4) * 3 - Key_state / 4 + 4; 594 switch (Mod_cur) { 595 case 0: 596 Mod_Time.HH = (H[0]*10 + num < 24)? H[0]*10 + num: 0; 597 break; 598 case 1: 599 Mod_Time.MM = (M[0]*10 + num < 60)? M[0]*10 + num: 0; 600 break; 601 case 2: 602 Mod_Time.SS = (S[0]*10 + num < 60)? S[0]*10 + num: 0; 603 break; 604 } 605 break; 606 } 607 } 608 if (Key_state < 4){ 609 disp_idx = 0; 610 n_shift = 0; 611 //flash erase 612 FLASH->KEYR = 0x45670123; // Unlocking sequence to access FLASH->CR register 613 FLASH->KEYR = 0xACEF59AB; 614 FLASH->CR = (1<<0); // PER = '1' : Page erase 615 FLASH->AR = FLASH_ADDR; //page to erase (page32 0x08008000) 616 FLASH->CR = (1<<6); // STRT = '1' : start erase operation 617 while(FLASH->SR & 0x1) {} // wait until FLASH operation complete. 618 FLASH->CR &= ~(1<<1); // PER = '0' 619 //flash programming 620 FLASH->CR = (1<<0); // PG = '1' : programming set 621 *((uint16_t*)FLASH_ADDR) = states; 622 while(FLASH->SR & 0x1) {} // wait until FLASH operation complete 623 FLASH->CR &= ~(1<<0); // PG = '0' : programming reset 624 FLASH->CR = (1<<7); // LOCK = '1' : lock FLASH->CR 625 EXTI->PR = 0x01; 626 } 627 } </pre>	<pre> 552 else if (mode == Illu_Displ) 553 TIM2->DIER &= ~(0x01); 554 switch (Key_state){ 555 case 0: 556 mode = Time_Displ; 557 states &= 0x0FFF; 558 states = (1 << 12); 559 break; 560 case 1: 561 mode = Temp_Displ; 562 states &= 0x0FFF; 563 states = (2 << 12); 564 break; 565 case 2: 566 EXTI->PR = 0x01; 567 return; 568 case 3: 569 EXTI->PR = 0x01; 570 return; 571 } 572 else if (mode == Time_Set) { 573 switch (Key_state){ 574 case 0: // Up 575 switch (Mod_cur) { 576 case 0: 577 Mod_Time.HH = (Mod_Time.HH + 1) % 24; 578 break; 579 case 1: 580 Mod_Time.MM = (Mod_Time.MM + 1) % 60; 581 break; 582 case 2: 583 Mod_Time.SS = (Mod_Time.SS + 1) % 60; 584 break; 585 } 586 EXTI->PR = 0x01; 587 return; 588 case 1: // Ignore 589 EXTI->PR = 0x01; 590 return; 591 default: 592 // 1 - 9 593 num = (Key_state % 4) * 3 - Key_state / 4 + 4; 594 switch (Mod_cur) { 595 case 0: 596 Mod_Time.HH = (H[0]*10 + num < 24)? H[0]*10 + num: 0; 597 break; 598 case 1: 599 Mod_Time.MM = (M[0]*10 + num < 60)? M[0]*10 + num: 0; 600 break; 601 case 2: 602 Mod_Time.SS = (S[0]*10 + num < 60)? S[0]*10 + num: 0; 603 break; 604 } 605 break; 606 } 607 } 608 if (Key_state < 4){ 609 disp_idx = 0; 610 n_shift = 0; 611 //flash erase 612 FLASH->KEYR = 0x45670123; // Unlocking sequence to access FLASH->CR register 613 FLASH->KEYR = 0xACEF59AB; 614 FLASH->CR = (1<<0); // PER = '1' : Page erase 615 FLASH->AR = FLASH_ADDR; //page to erase (page32 0x08008000) 616 FLASH->CR = (1<<6); // STRT = '1' : start erase operation 617 while(FLASH->SR & 0x1) {} // wait until FLASH operation complete. 618 FLASH->CR &= ~(1<<1); // PER = '0' 619 //flash programming 620 FLASH->CR = (1<<0); // PG = '1' : programming set 621 *((uint16_t*)FLASH_ADDR) = states; 622 while(FLASH->SR & 0x1) {} // wait until FLASH operation complete 623 FLASH->CR &= ~(1<<0); // PG = '0' : programming reset 624 FLASH->CR = (1<<7); // LOCK = '1' : lock FLASH->CR 625 EXTI->PR = 0x01; 626 } 627 } </pre>
--	--

앞서 언급한 대로, Mode 별 key입력에 따른 동작을 수행한다. Time_Set 모드를 제외한 나머지 모드에서는 Row[0]의 key만 입력받으며, 자기 자신의 mode를 선택한 경우 별다른 동작 없이 EXTI0 Handler가 종료된다. 그렇지 않을 경우, n_digit, n_shift를 초기화 하여 display가 처음부터 scrolling 될 수 있도록 하고, FLASH에 states변수를 저장한다. States 변수의 bit당 역할은 하단의 표에 정리하였으며 Bit[15:8]은 설정저장용, Bit[7:0]은 Dot matrix 출력용으로 구분하였다. Time_Set 모드에서는 모든 key입력이 가능하며, UP(key_idx = 0) & DOWN(key_idx = 2) 및 숫자 keypad로 시간을 설정하게 된다. 이후 Confirm(key_idx = 3) 버튼을 누르면 시간설정 모드에서 설정한 시간인 Mod_Time을 Base_Time으로 Update하게 된다. Key 별 동작은 단계별 구현방법에서 서술한 내용과 같다.

Bit[15]	Bit[14]	Bit[13]	Bit[12]	Bit[11]	Bit[10]	Bit[9]	Bit[8]
Mode - Time_Displ	Mode - Temp_Displ	Mode - Illu_Displ	Reserved	시간모드 0 - 12시제 1 - 24시제	온도모드 0 - 섭씨 1 - 화씨	시간설정 0 - 미설정 1 - 설정	Reserved
Bit[7]	Bit[6]	Bit[5]	Bit[4]	Bit[3]	Bit[2]	Bit[1]	Bit[0]
Reserved					오전/오후	12시/24시	Blink

USART1 ISR	
<pre> 537 void USART1_IRQHandler(void) { // CTS interrupt -> Send next 538 if (USART1->SR & 0x01) { // TXE == '1' : TXR empty 539 n_transmit++; 540 if (n_transmit < 8){ 541 USART1->DR = (IsConnect)? tx_data[n_transmit]: TEST_DATA; // Send next data 542 } 543 USART1->SR &= ~(0x0200); 544 } 545 } </pre>	nCTS 입력pin에 변화가 있을 시 발생하는 인터럽트로 한 번의 RTS 신호가 인가되면 n_trasmit이 1 증가하여, 각 Cycle에서의 데이터 전송횟수를 확인할 수 있다. 연결이 불량할 경우 테스트 데이터(0)을, 그렇지 않을 경우 다음 tx_data를 송신한다.

ii) Board2

Main function	
<pre> 92 /*-----Main function-----*/ 93 u8 i; 94 while(1) { 95 _WFI(); 96 if (mode != Disp_On) 97 continue; 98 for (i = 1; i < 6; i++){ 99 display[0] = (display[0] << 4) + font5x4[rx_data[i]][0]; 100 display[1] = (display[1] << 4) + font5x4[rx_data[i]][1]; 101 display[2] = (display[2] << 4) + font5x4[rx_data[i]][2]; 102 display[3] = (display[3] << 4) + font5x4[rx_data[i]][3]; 103 display[4] = (display[4] << 4) + font5x4[rx_data[i]][4]; 104 } 105 display[0] = display[0] << rx_data[i]; 106 display[1] = display[1] << rx_data[i]; 107 display[2] = display[2] << rx_data[i]; 108 display[3] = display[3] << rx_data[i]; 109 display[4] = display[4] << rx_data[i]; 110 } 111 } </pre>	Disp_on mode 상태에서는 계속해서 Dot matrix display에 received data를 의미하는 rx_data에 해당하는 font를 load시켜 board1로부터 수신한 rx_data를 board2 dot matrix display에 표시하게 해준다. 계속해서 같은 동작을 하면서 다른 interrupt가 나타날 때까지 기다린다.
TIM1 UPDATE ISR	

<pre> 114 void TIM1_UP_IRQHandler(void) { // 8Hz - Blinking & brightness control 115 if((TIM1->SR & 0x01) != 0) { 116 // Display brightness control 117 if (mode == Disp_On) 118 TIM1->CCR2 = (rx_data[0] * 550) / 255 + 40; 119 // LED blinks 120 else{ 121 GPIOA->ODR ^= (1u << 5); 122 } 123 TIM1->SR &= ~(0x01u); 124 } 125 } </pre>	TIM1 update event frequency가 8Hz로 설정되어 있는 상태이며, TIM1 update event interrupt가 발생할 때마다, mode가 Disp_on 이면, brightness 정보를 담고 있는 rx_data[0] 를 TIM1 capture/compare register 2에 load하여 dot matrix의 row를 active low 하는 시간을 조절하여 board2의 밝기를 control 하게 된다. Disp_on mode가 아닐 때는, PA5에 해당하는 LED를 on일 때는 off 동작을, off일 때는 on 동작을 하도록 설정하여, 1초에 4번씩 LED가 깜빡이도록 설정하였다.
TIM1 CAPTURE/COMPARE ISR	
<pre> 127 void TIM1_CC_IRQHandler(void) { // Dot matrix scanning 128 static u8 d_row = 0; 129 static u16 d_col; 130 if((TIM1->SR & 0x02) != 0){ 131 if (d_row < 5){ 132 GPIOC->ODR &= ~(0xFF); 133 GPIOC->ODR = (~(!<d_row))&0x00FF; 134 d_col = (display[d_row] >> 4); 135 GPIOB->ODR = d_col; 136 } 137 else if (d_row < 7){ 138 GPIOB->ODR = 0; 139 } 140 else if (d_row == 7){ 141 GPIOC->ODR &= ~(0xFF); 142 GPIOC->ODR = (~(!<d_row))&0x00FF; 143 GPIOB->ODR = rx_data[7]; 144 ADC1->CR2 = (0x1 << 22); 145 } 146 d_row++; 147 if (d_row >= 8) 148 d_row = 0; 149 TIM1->SR &= ~(0x02); 150 } 151 if ((TIM1->SR & 0x04) != 0){ 152 GPIOB->ODR = 0; 153 TIM1->SR &= ~(0x04); 154 } </pre>	TIM1 capture/compare 1 interrupt 은 TIM1_CNT=0 일 때마다 발생하며, 120Hz 마다 dot matrix scanning 을 위해 사용된다. d_row<5 인 영역에서는 dot matrix row 0~4 중 다음 row에 해당하는 row를 active low로 enable 시켜주면서 dot matrix scanning을 진행한다. d_row 가 5,6 일 때는 정보를 표시하지 않기 위해 column을 모두 off 시켜주며, d_row가 7이면, state정보를 담고 있는 rx_data[7] 을 column에 표시해준다. TIM1 capture/compare 2 interrupt 은 brightness 정보에 의해 TIM1->CCR2 레지스터에 쓰여진 정보에 따라 dot matrix display가 꺼지게 만들어, dot matrix display의 밝기가 update되도록 해준다
USART1 ISR	
<pre> 202 else if (mode == HandShake){ 203 if (data){ 204 n_receive = 0; 205 rx_data[n_receive++] = data; 206 mode = Disp_On; 207 GPIOA->ODR = 0x20; // LED On 208 TIM1->DIER = 0x07; // TIM1_CH1&2 interrupt enable 209 } 210 } 211 else { 212 rx_data[n_receive++] = data; 213 n_receive %= 8; 214 } 215 } 216 } 217 </pre>	USART interrupt가 발생하면, 유효한 data가 아닐 경우 frame error가 발생하므로, 이 때 mode를 disconnect 로 설정하고, Dot matrix scanning과 brightness update를 담당하는 tim1 ch1,2를 disable 시키고, 연결이 안정됨을 표시하는 lsStable=0, RTS를 DISABLE 시켜주며, dot matrix display를 off시킨다. 유효한 data가 수신되면, 연결의 안정성을 위해 n_receive가 6일 때까지 통신을 진행하고, 그 이후에는 동작의 안정성이 확보되었음을 확인하고, handshake mode로 설정하며, RTS를 enable 시켜준다. Handshake mode에서 수신 받은 data를 rx_data로 load하며, mode를 Disp_on 으로 설정해주며, PA5 LED를 ON시켜 준다.